ENTWINE — Experimentation Kickoff Instructions

Purpose: get the team from "we understand the architecture" to "we have working, reproducible results on real data" before touching production-scale build-out. This mirrors the single-building proof-of-concept used for the panel — that's deliberate. If it worked to prove feasibility to reviewers, it's the right first milestone for the team too.

Phase 0 — Environment setup (all students, before anything else)

1. Set up a shared Python environment (`venv` or `conda`) with a pinned `requirements.txt` — everyone on the same versions from day one avoids "works on my machine" losses later.
2. Install PostgreSQL locally (or via Docker) and load the `asset_registry_schema.sql` provided — every student should be able to stand this up independently, not rely on one shared instance yet.
3. Set up a shared Git repository with this structure:

```
entwine/
  registry/        # asset registry schema + seed scripts
  ingestion/        # state-layer data loading
  models/            # GridReason / GrCF / CAFA integration
  forecasting/        # 3-month load prediction (later phase)
  interrogation/      # agentic/RAG layer (later phase)
  dashboard/           # frontend (later phase)
  logs/                 # experiment log (see Phase 3)
```

4. Confirm every student can run the existing GridReason / GrCF / CAFA code end-to-end on the KCT Powerhouse dataset **before** any new code is written. This is a checkpoint, not busywork — if the baseline doesn't reproduce, nothing built on top of it should proceed.

Phase 1 — Single-building reproduction (Weeks 1–2)

Goal: reproduce the validated results from your own research, inside the ENTWINE structure, on one building (PH-01).

1. Populate the asset registry with PH-01's real data (building metadata, meter info) — this is the seeded example already in the schema; extend it with real equipment inventory if available.
2. Load the historical KCT Powerhouse dataset into the state layer (TimescaleDB) instead of live ingestion — this removes the meter-access dependency from the critical path.
3. Run GridReason against PH-01's state-layer data and confirm the anomaly-detection output matches (or is consistent with) the validated F1 = 0.9524 result.

4. Run GrCF against a flagged anomaly and confirm a counterfactual explanation is produced.
5. Run CAFA's audit and confirm the ρ>1 applicability check and recall-disparity output reproduce.

Checkpoint: if all three models run cleanly against data pulled from the twin's own state layer (not directly from the original dataset files), Phase 1 is done. This is the technical proof that "twin-shaped data" is a valid substrate for the models — worth treating as a real go/no-go gate before Phase 2.

Phase 2 — Scale to remaining buildings (Weeks 3–5)

1. Extend the asset registry to all 8 meters.
2. Batch-load historical data for each into the state layer.
3. Run the same model pipeline per building, on a schedule (introduce the scheduler here — `APScheduler` is enough at this stage).
4. Compare results across buildings — this is where cross-building consistency (or lack of it) becomes visible, and it's worth logging carefully (see below).

Phase 3 — Keep an experiment log from day one

Every student should log, per experiment run:

- What was run, on what building, against what data window
- The result (metric values, flagged anomalies, ρ value)
- **Anything that diverged from the validated baseline** — this is the most valuable line in the log, not a failure to hide. Divergences between offline-validated results and twin-integrated results are exactly the material a deployment paper would need later, and exactly what a review panel will ask about.

A simple shared spreadsheet or markdown log per week is enough — don't over-engineer this part.

What NOT to start yet

To keep the team from overreaching before the foundation is solid:

- **No live meter ingestion** until Phase 1–2 are working on historical data.
- **No dashboard work** until there's real twin output to visualize — build against mock data only if a visual is needed for an interim review.
- **No agentic/RAG layer** until the state and model layers are producing reliable, queryable output — there's nothing meaningful to query before that.

Weekly check-in structure (suggested)

- **15 min per student**: what ran, what broke, what's logged

- **Blockers flagged early** — especially anything touching meter data access, since that's the one external dependency the team doesn't control
- **Log review** — you (or a senior student) spot-check the experiment log weekly to catch silent divergences before they compound